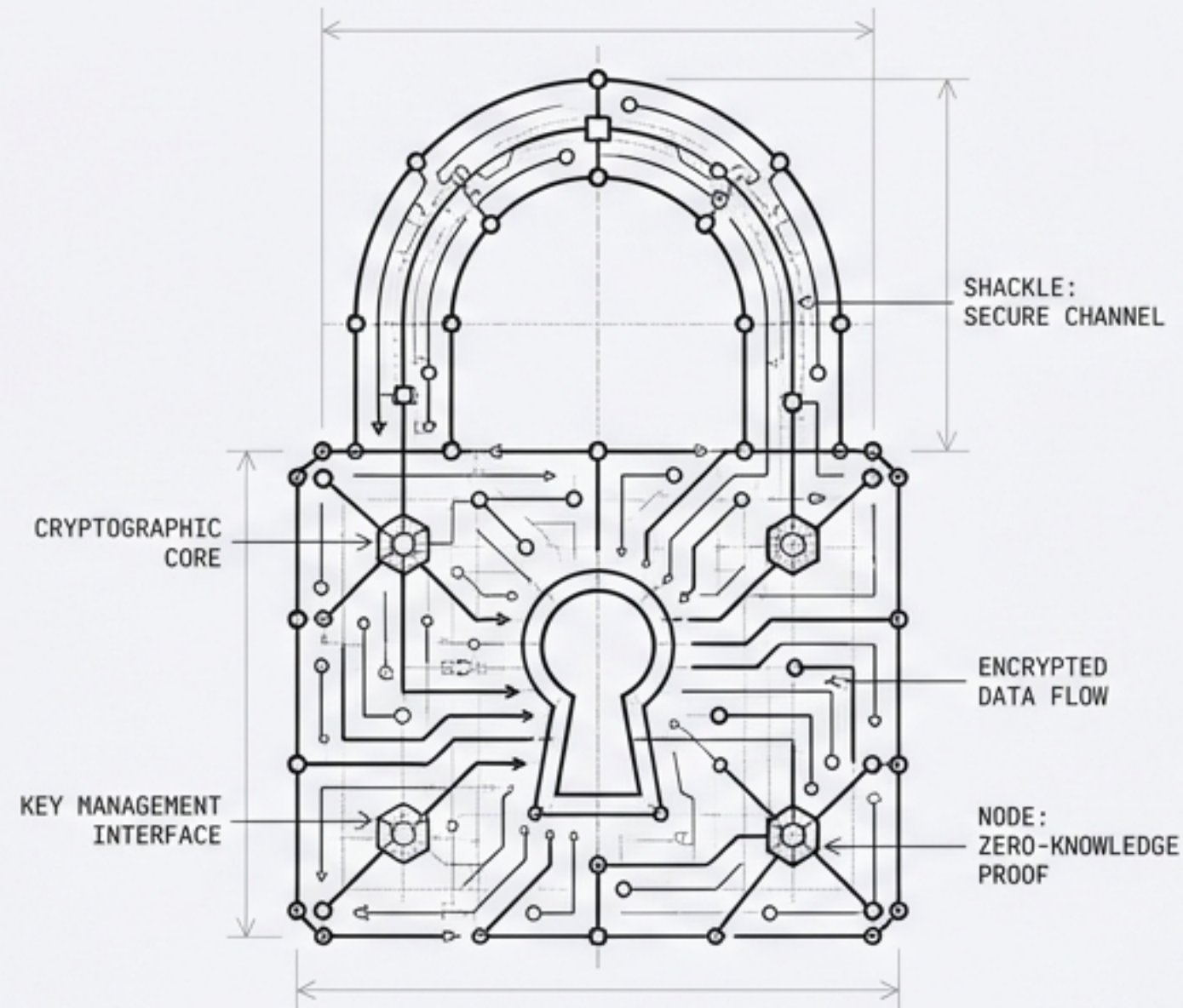




Status: LIVE – v0.4.25



PKI Messaging System: Architecture Briefing

A zero-knowledge, client-side cryptographic communication layer.

The Server is a Dumb File System

Traditional Model



Server sees all.

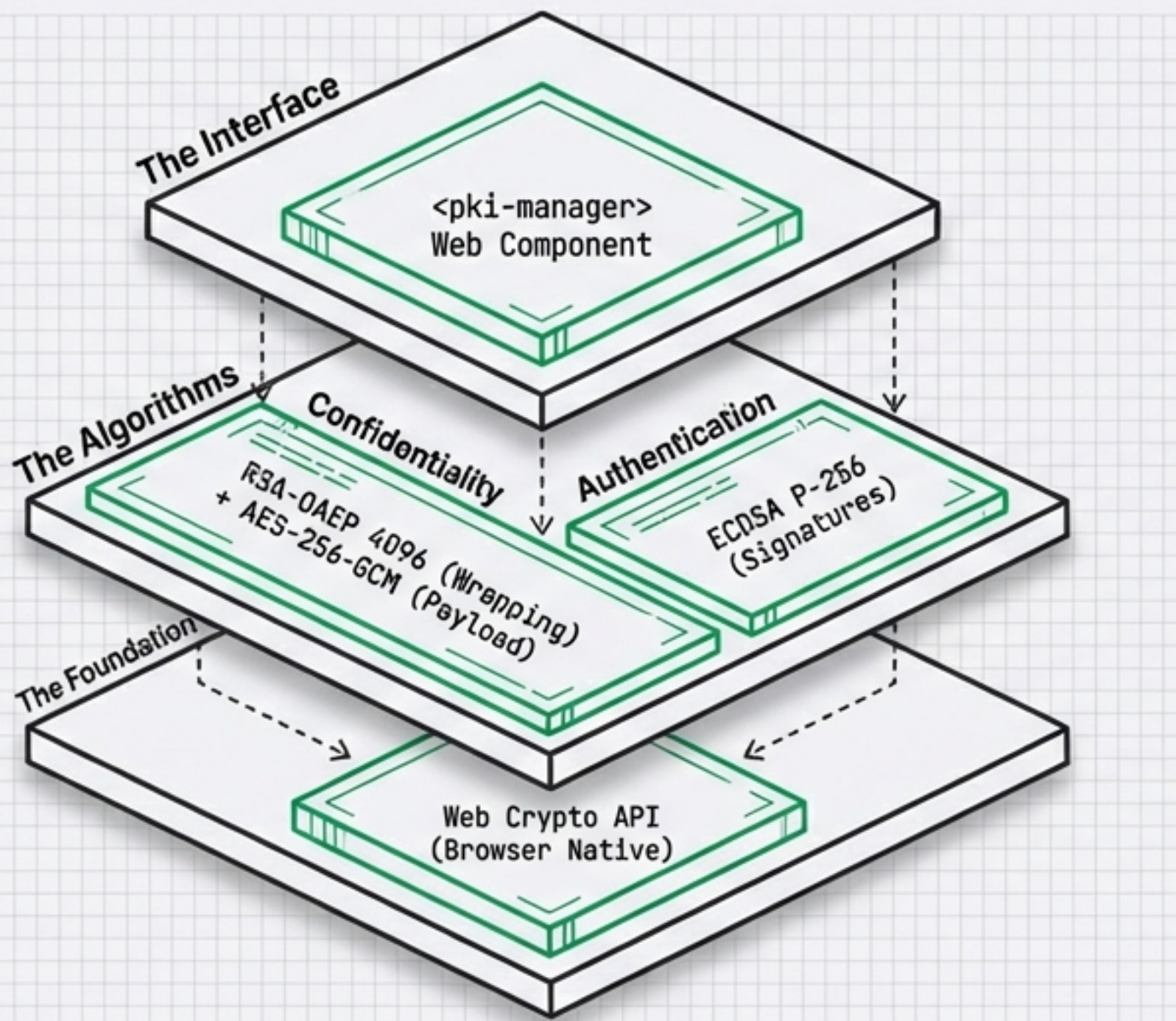
Zero-Knowledge Architecture



Server sees static HTML only.

The server serves the static HTML/JS application but is mathematically incapable of reading the message content. Confidentiality and Sender Authentication are enforced exclusively in the browser.

Standard Primitives. No Custom Crypto.

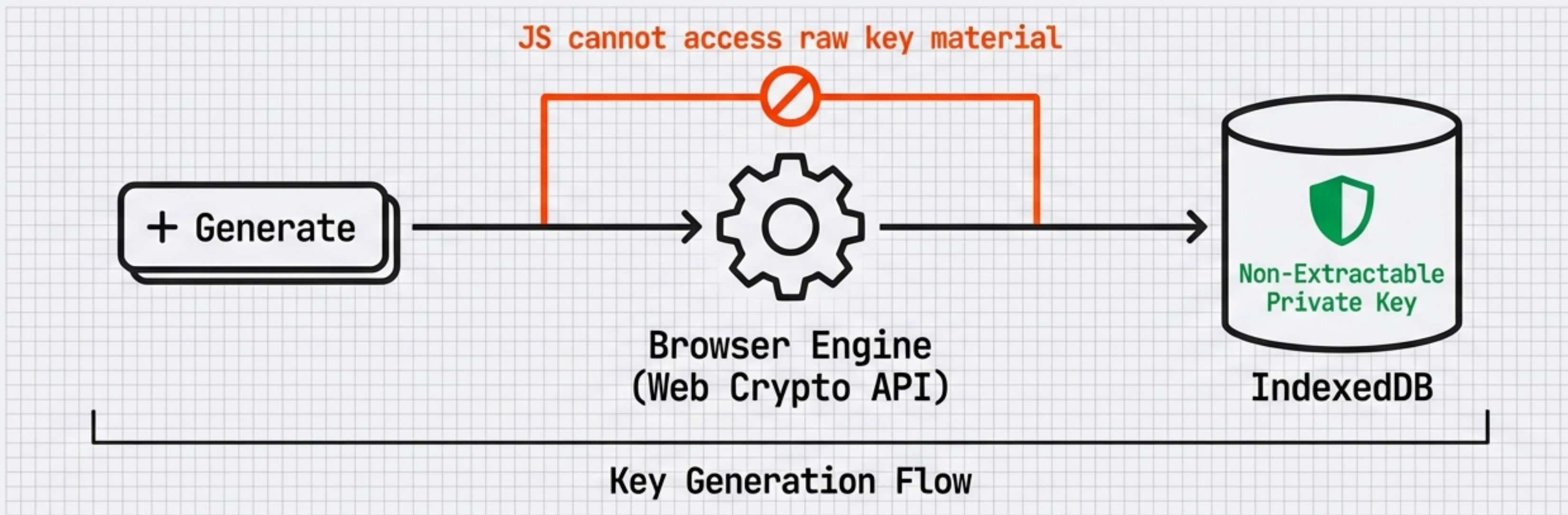


We rely on browser-native, non-extractable key storage.

We combine the speed of AES with the security of RSA and the authenticity of ECDSA.

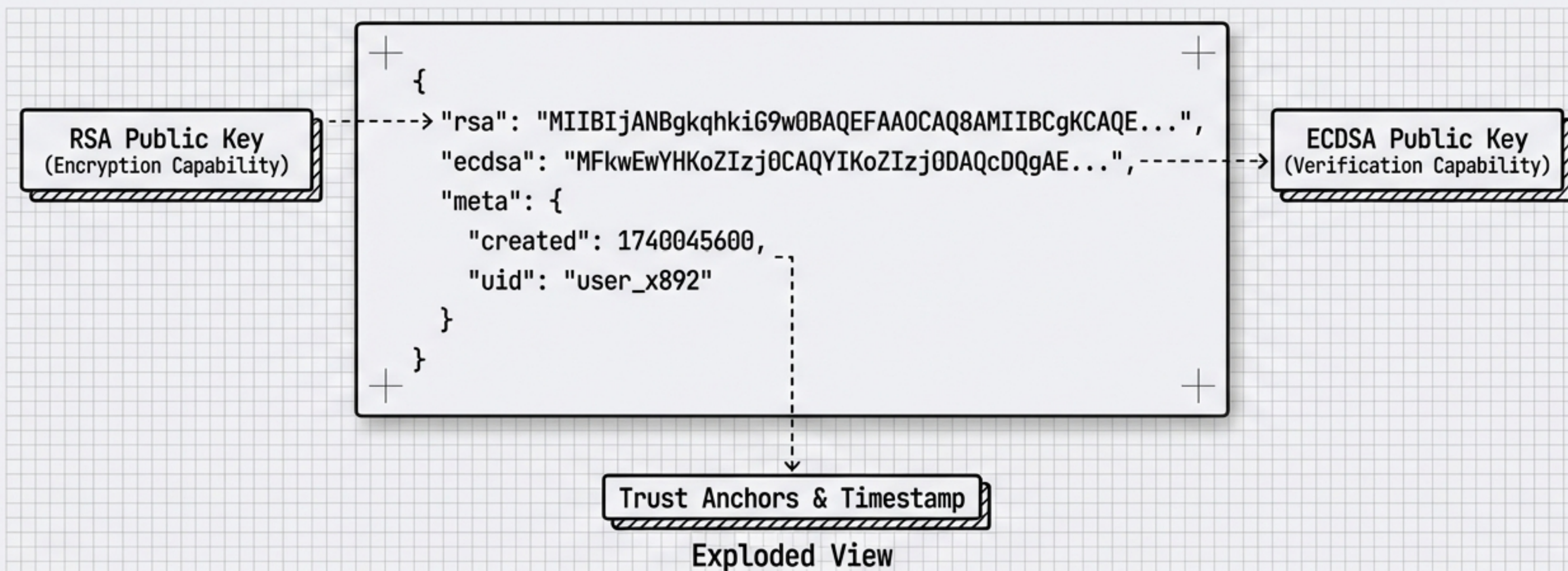
Hardware-Backed Security.

When a user generates a key, it never leaves the browser engine. Even our own JavaScript cannot read the raw private key material—only the Web Crypto API can access it for operations.



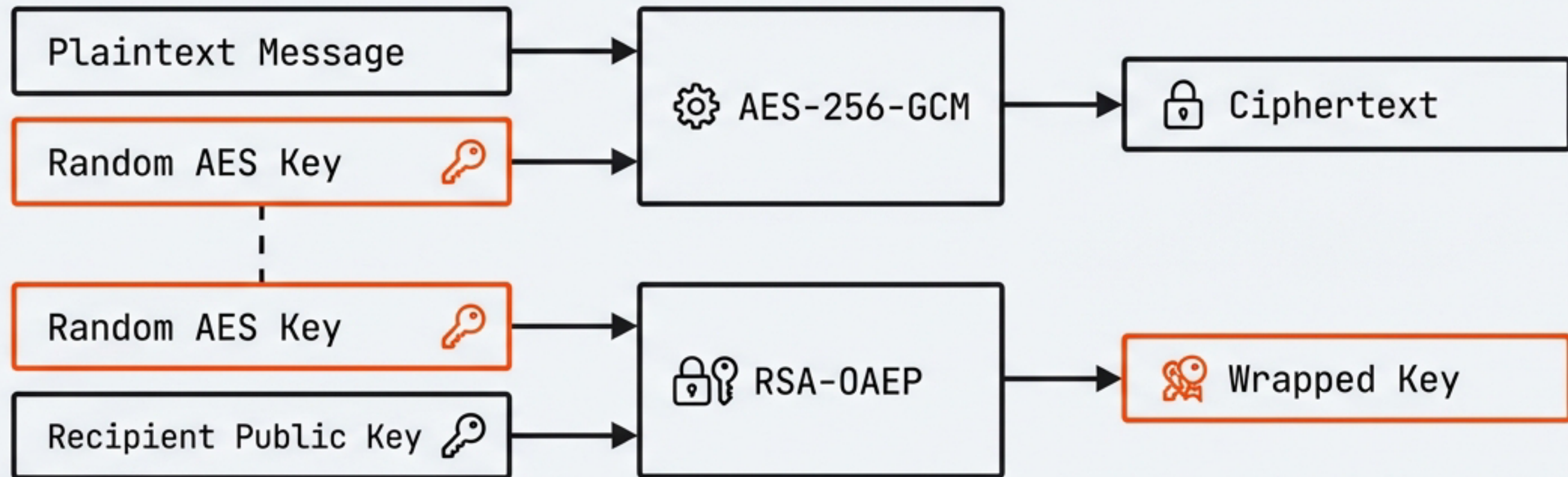
The Identity Card.

Identity is portable. When a user clicks 'Copy Public Key', the component builds a JSON bundle containing both encryption and signing capabilities. On import, the system auto-detects the format to establish the trust anchor.



Workflow Part 1: Hybrid Encryption

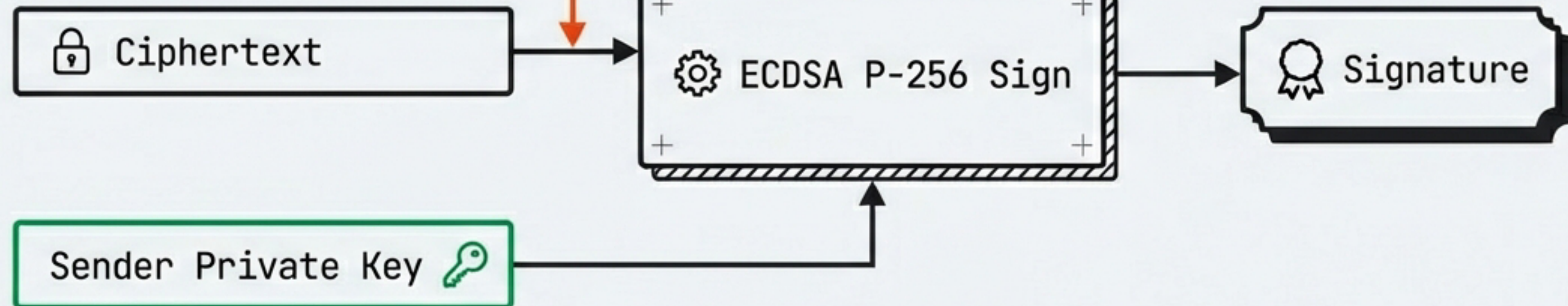
We solve the performance bottleneck of RSA by using it only to wrap a transient AES key. The message itself travels inside a symmetric tunnel using AES-GCM.



Workflow Part 2: Signing the Envelope.

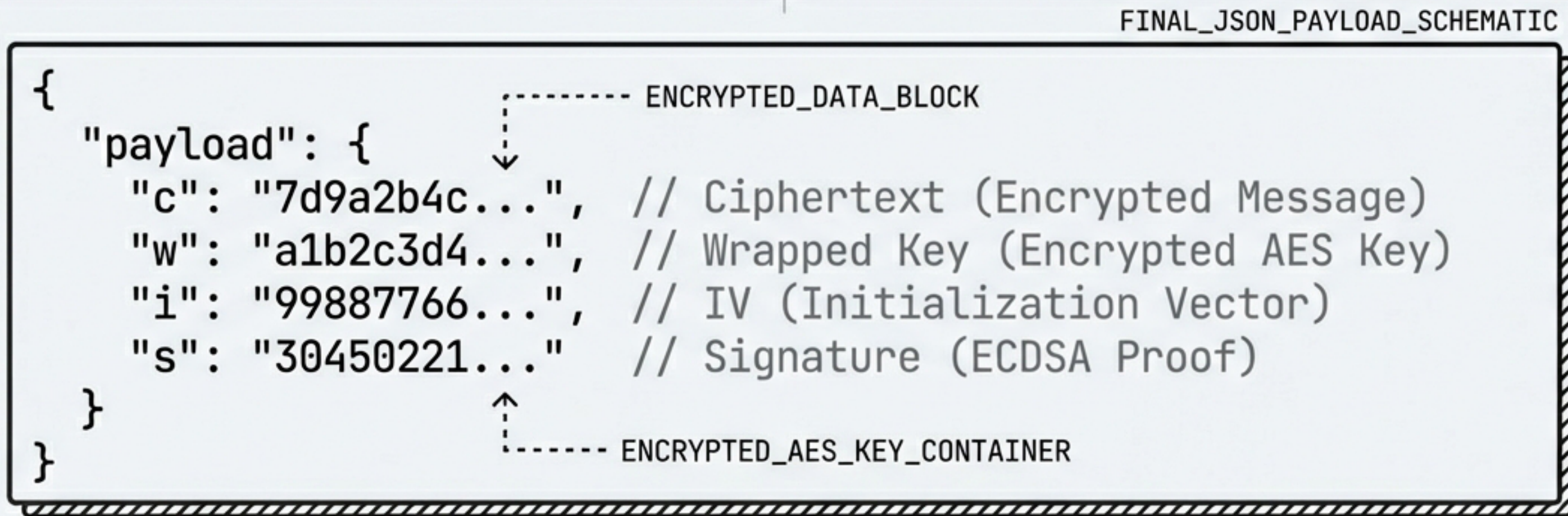
We sign the ciphertext, not the plaintext. This creates a cryptographic proof that the sender possessed the private key, ensuring integrity before the recipient even attempts to decrypt.

We sign the encrypted bytes, NOT the plaintext.



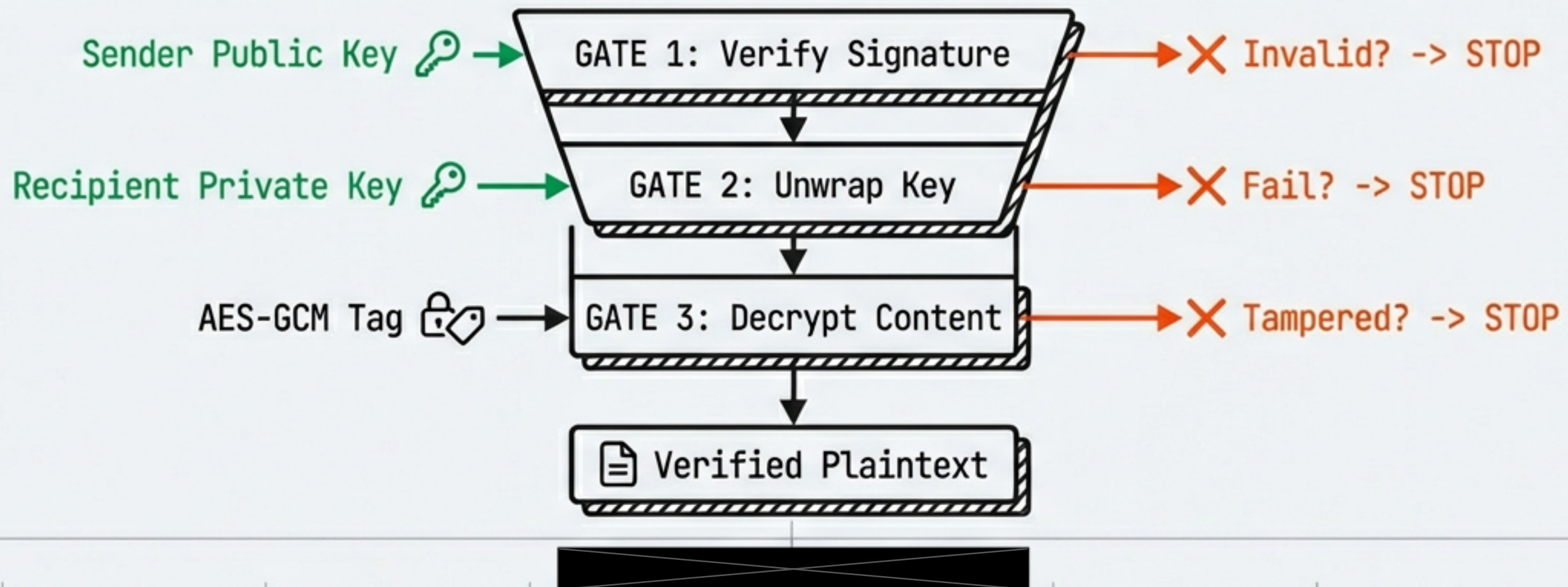
What the Network Sees.

This is the only data that leaves the client. To the outside world, it is opaque high-entropy noise. The server cannot interpret a single byte of this object.



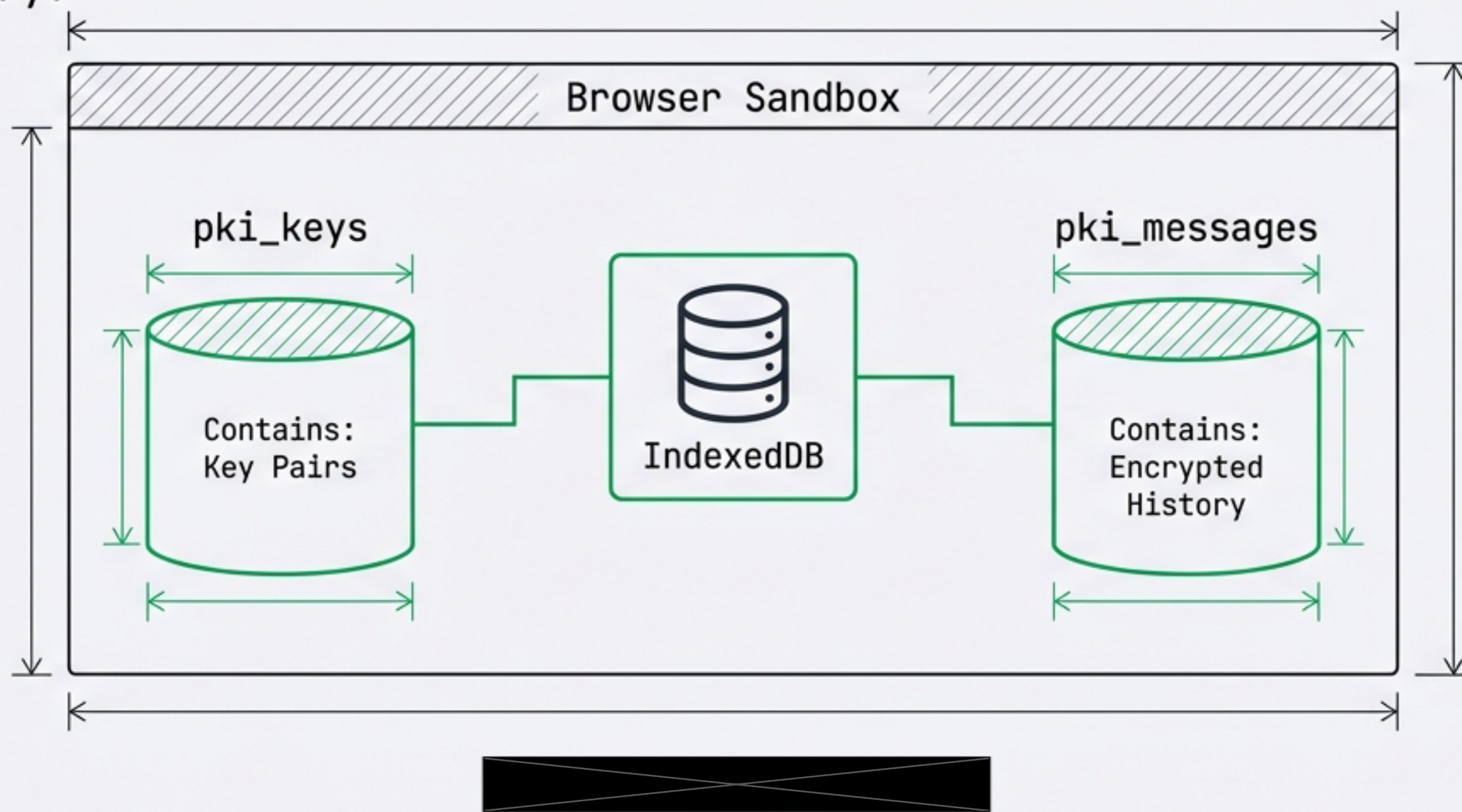
Trust, then Verify.

The system executes a strict verification chain. If the ECDSA signature doesn't match, or the AES integrity tag is invalid, the operation aborts immediately. We never show the user unverified data.



Local-First Persistence.

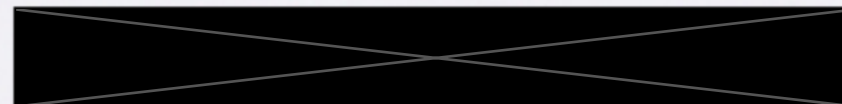
Keys and messages persist in the user's browser database (IndexedDB). This ensures the application state is maintained between sessions without a single server query for message history.



The Security Promise.

We provide strong confidentiality and integrity. We are transparent about the lack of forward secrecy (keys are reused) and the partial nature of non-repudiation.

Security Properties		
Property	Status	Mechanism / Reason
Confidentiality	Yes	RSA-OAEP + AES-GCM
Integrity	Yes	AES-GCM Auth Tag
Sender Auth	Yes	ECDSA over ciphertext
Non-repudiation	Partial	Key possession proof, no timestamp
Forward Secrecy	No	Static RSA key pair reuse



Self-Contained Excellence.

The entire system is encapsulated in a single Web Component (<pki-manager). No external libraries, no 'npm install' supply chain risks. Just clean, audit-able vanilla JavaScript.

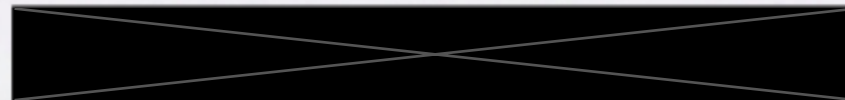
root/

└─ components/

└─ pki-manager/

└─ **pki-manager.js** →

Lines of Code: 1821
Dependencies: 0



Ready for Secure Communication.

Status: LIVE

The infrastructure is silent; the browser is the fortress. The system is live and operating within the defined security parameters.



v0.4.25 Deployed to Production Admin Console.

Architect: Team Explorer